删除链表头节点，Linus 只用了一个二级指针：好品味就是消灭特例

嵌入式 Linux 手记　2026-10-18　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。这段"教科书级"的按值删除，删头节点和删中间节点都能工作：

```
struct Node* remove_val(struct Node* head, int v) {
    struct Node* prev = NULL;
    struct Node* cur  = head;
    while (cur) {
        if (cur->data == v) {
            if (prev) prev->next = cur->next;
            else      head = cur->next;
            free(cur);
            return head;
        }
        prev = cur; cur = cur->next;
    }
    return head;
}
```

功能没问题，每一行也都能讲出道理：删中间节点要改前驱的 next，删头节点没有前驱，只能改 head 本身，于是有了 `if (prev)` 这道分叉。但盯着这道分叉多看两秒，一个不舒服的感觉会浮上来：删头节点和删中间节点，分明是同一件事，凭什么要分两条路写？

Linus 在一次公开访谈里把这种感觉叫作"品味"（taste）的缺失，举的例子正是这段代码。他的版本只有一个游标：

```
struct Node* remove_val2(struct Node* head, int v)
{
    struct Node** indirect = &head;
    while (*indirect != NULL && (*indirect)->data != v)
        indirect = &(*indirect)->next;
    if (*indirect != NULL) {
        struct Node* dead = *indirect;
        *indirect = dead->next;
        free(dead);
    }
    return head;
}
```

那道 `if (prev)` 特例不见了，头节点和中间节点走的是一条路。这一篇拆解它凭什么能做到，以及它比"少一个 if"多给了什么。

一、特例的根源：两处指针，一种关系

先看清旧写法为什么非分叉不可。链表里"摘掉一个节点"这个动作，本质只有一句话：把指它的那个指针，改成指向它的后继。

问题在于，指着头节点的指针是 `head`（一个指针变量），指着中间节点的指针是 `prev->next`（一个指针成员）。名字不同、写法不同、一个在栈上一个在堆上，旧写法于是把同一句话拆成了两句：`head = cur->next` 和 `prev->next = cur->next`，外面再包一层 if 决定用哪句。

可如果跳出"变量名"的层面，单看类型和关系，`head` 与 `prev->next` 完全同构：两者的类型都是 `struct Node*`，两者的值都指向当前节点，两者都是"当前节点的上游指针"。差别只在身份，不在结构。同构的两样东西被迫用两套写法处理，这就是特例的全部根源。

Linus 的改写，就是把"上游指针"这个共同身份从变量背后提到前台：让游标不记"节点在哪"，改记"指着节点的那个指针在哪"。这就是 `indirect` 三个字的含义。

二、indirect 游标：一个位置，三种场景

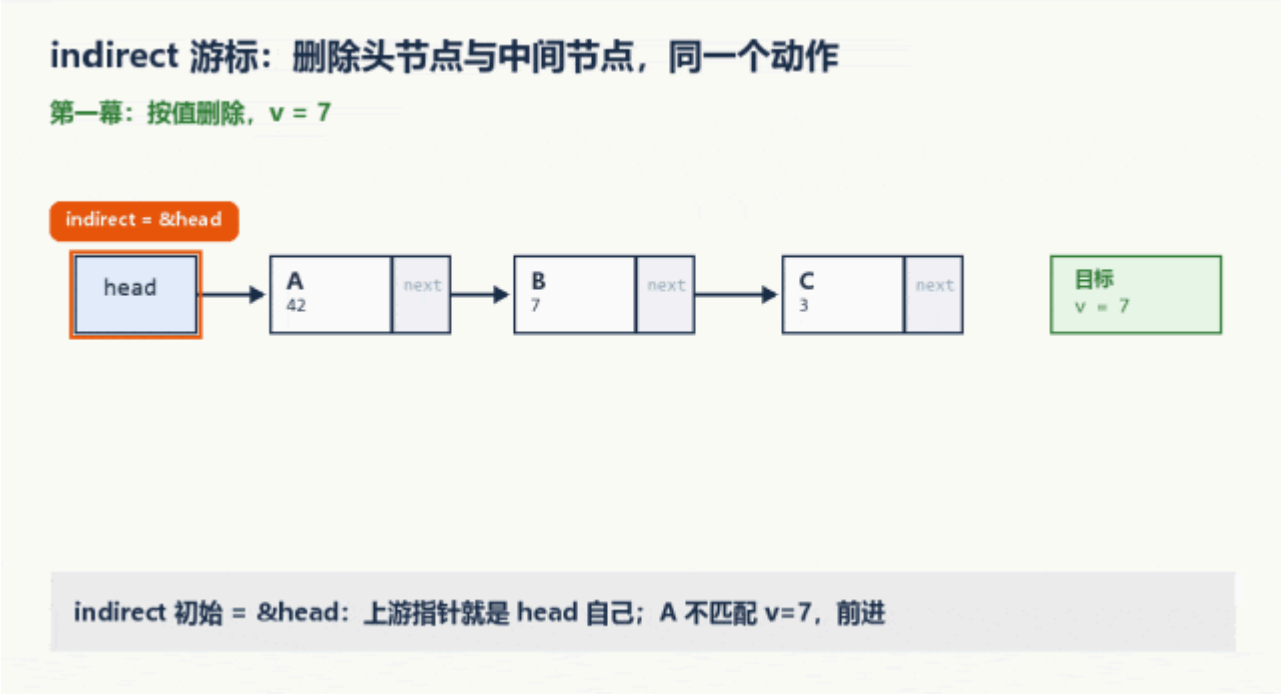
`indirect` 的类型是 `struct Node**`，它的值是某个 `struct Node*` 的地址。这个"某个"可以是 `&head`，也可以是 `&prev->next`，两种场景共用同一个游标，这是全部魔法的来源。

`*indirect` 是当前节点；`indirect` 是"指着当前节点的那个指针"的地址。删当前节点只有一句：`*indirect = dead->next`。删头节点时 `indirect` 恰好等于 `&head`，这句改的就是 `head` 本身；删中间节点时它等于 `&prev->next`，这句改的就是前驱的 next。同一个动作，自动适配两种场景，if 就此消失。

还有一句：函数结尾为什么还要 `return head`？`head` 是形参副本，被 `*indirect` 改写后，新表头必须交还调用者，这正是篇 4 说的两条正路之一："返回新值"而非"改形参本身"。

游标怎么前进？`indirect = &(*indirect)->next`：先解引用拿到当前节点，再取它 next 成员的地址。注意这一步取的不是 next 的值，而是 next 这个指针自己所在的地址，游标永远站在"上游指针"的位置上，而不是站在节点上。这是和篇 3 两级指针一脉相承的读法：上层格子

（`indirect` 存的地址）与下层对象（`*indirect` 指向的节点）各归各的，只是这里上层的格子里装的是 `next` 成员的住址。



图：indirect 游标推进与删除动画：游标始终站在上游指针的位置，删除动作对头节点与中间节点完全一致

三种边界场景走一遍，确认没有藏着的新特例：

1. 空链表：`*indirect` 一开始就是 NULL，循环不进，if 不进，直接返回原 head。
2. 找不到目标值：游标推进到链表末尾，`*indirect` 变 NULL，循环退出，if 不执行，链表原样返回。
3. 删除尾节点：`dead->next` 是 NULL，`*indirect = NULL` 顺手把新尾节点的 next 置空，不需要任何额外判断。

尾节点场景值得多想一秒：旧写法在这里其实也没特例（`prev->next = NULL` 顺手成立），但新写法把这份"顺手"也纳入了同一句式。好结构的标志就在这里：不仅消灭显眼的特例，连隐性的边界也一并收编。

三、可读性代价：两级解引用怎么读

诚实地说，这段代码是有门槛的。`indirect = &(*indirect)->next` 一行里出现两次解引用、一次取址、一次成员访问，没练过的人要盯十秒。这份门槛是真实的成本，Linus 自己也不否认，他的态度是：这段代码初看吓人，想通之后你会发现它反而更简单。判断标准因此不是"好不好读"，而是"想通之后的世界里，它是否更简单"。

给出一个可操作的读法，两层各读各的：

- 第一层读关系：`indirect` 是个住址，住址里住着一个 `struct Node*`，那个指针指着"当前节点"。
- 第二层读动作：`*indirect = dead->next` 翻译成人话就是"让指着 dead 的那个指针，改为指向 dead 的后继"，和篇 4 里 `*head = n` 写穿到调用者的指针，是同一招的链表内化版。

想验证自己真懂了，一个简单的测试：把 `while` 循环条件换成"找到第一个偶数就停"，不动其他行，代码应该依然成立。成立的原因在于循环体根本没有依赖"奇数还是偶数"的任何假设，游标只负责走，删除只负责改上游指针，两个职责都足够薄。

什么时候不值得上这套？删一个节点、写完就扔的教学代码，`prev` 版更直观，没必要为一行 if 付出阅读成本；团队多数人不熟悉间接指针时强行推广，省下的 if 会变成成倍的 review 摩擦。它是"删多个值""边遍历边删""通用删除接口"这些场景的利器，前提是代码会被反复读。

四、隐藏红利：边遍历边删除，不用临时变量

旧写法还有一处常被忽略的丑：如果要删除所有值为 v 的节点（不止第一个），循环里必须处理"删除后 cur 怎么走"。删除动作改了 `cur->next` 的归属，`cur` 下一个该指向谁，要么在删除前存临时变量，要么把推进逻辑也拆出分支。写出来是这样的：

```
while (cur) {
    struct Node* next = cur->next;    /* 删之前先存好 */
    if (cur->data == v) { ...free(cur); }
    cur = next;
}
```

临时变量 `next` 是不得不交的税——篇 8 讲过，`free(cur)` 之后再读 `cur->next` 就是悬空指针。间接指针把这个税也免了：删除时 `*indirect = dead->next` 已经让上游指针指向后继，游标原地不动就是站在后继面前；没删才需要前进。一套"删则不前进、不删则前进"的语义，遍历和删除彻底解耦：

```c
/* 删除所有满足 pred 的节点，遍历与删除一次完成 */
void remove_if(struct Node** head, int (*pred)(struct Node*)) {
    struct Node** indirect = head;
    while (*indirect != NULL) {
        struct Node* entry = *indirect;
        if (pred(entry)) {
            *indirect = entry->next;    /* 删除：游标原地不动 */
            free(entry);
        } else {
            indirect = &entry->next;    /* 保留：前进一步 */
        }
    }
}
```

这才是"少一个 if"背后真正值钱的东西：旧写法把"怎么走"和"删不删"两件事缠在一起，每加一种删除策略就多一圈纠缠；新写法把它们拆开，策略全部关进 `pred` 回调里，遍历逻辑从此不管删什么、删几个。回调的用法篇 6 拆过，这里它第一次替链表算法打工。

五、这不是玩具：内核把间接指针物化进了数据结构

有人觉得 indirect 是面试层面的小聪明。内核社区的态度相反：他们嫌"现场构造 `&(*indirect)->next` "还不够，直接把间接指针存进了节点里。

单向链表的表头嵌在内核对象里的形态是 `struct hlist_node`，比普通链表节点多一个字段：`pprev`。它存的正是"指着我的那个 next 指针的地址"——要么是上一个节点的 `&prev->next`，要么是表头的 `&first`。删除时不需要遍历找前驱，不需要特判头节点，`hlist_del` 一行收工：

```c
*n->pprev = next;                   /* 上游指针改指后继 */
if (next) next->pprev = n->pprev;   /* 后继认新来路（尾节点无后继） */
```

看清这组设计：`pprev` 就是物化的 `indirect`，"消除头节点特例"从写法技巧上升为数据结构约定。Linus 的品味之争从来不是少写一个 if 的小账，而是"要不要让共同结构浮出水面"的大账。`hlist` 用一整个字段投了票。

最后校准一个流行误区：间接指针不是万能钥匙。双向链表（`struct list_head`）删除靠 `prev` / `next` 一对指针即可 O(1) 完成，`list_del` 不需要间接指针也不需要找前驱，它靠的是每

个节点都认得来路。单向链表里没有这份记忆力，间接指针就是替代方案。选哪种，取决于你愿意为节点多付 8 字节，还是为读者多付一层解引用。

写在最后

1. 特例的根源是同构关系被迫用两套写法：head 与 prev->next 同为"上游指针"，旧写法却让它们走两条路。
2. indirect 游标记住的不是节点位置，而是"指着节点的指针的位置"；*indirect = dead->next 一个动作覆盖头节点、中间节点、尾节点三种场景。
3. 游标前进取的是 next 成员的地址而非值：indirect = &(*indirect)->next，这是它始终站在上游指针位置的原因。
4. 隐藏红利是边遍历边删除：删除不推进、保留才前进，临时变量税一并免掉，remove_if 把策略收进回调。
5. 内核 hlist 的 pprev 字段把间接指针物化进数据结构，是对这套思路最硬的背书；双向链表有 prev 指针时不必用它，选型看内存与读者的账怎么算。

链表里的指针游戏讲到这，指针还压着一块更深的雷：`int (*p)[3]` 和 `int **p` 长得像父子，传参时却一个对一个错。下一篇《二维数组传参写成 int** 会怎样》，把篇 2 的四要素、篇 3 的步长和篇 1 的退化一次性收束。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。